



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SEZG651/ SSZG653 Software Architectures

CS03 Quality Attributes 2 of 2



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Interoperability Module 2 – L3

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs



SEZG651/ SSZG653

Software Architectures

Module 2-CS03C

Credits



These Slides are based on

- Software Architecture in Practice by
 - Len Bass, Paul Clement and Rick Kazman
 - Pearson © 2013

Outline



- What is Interoperability?
 - Interoperability General Scenario
- Tactics for Interoperability
 - A Design Checklist for Interoperability
- Summary

What is Interoperability?



- Interoperability is about the degree to which two or more systems can usefully exchange meaningful information.
- Like all quality attributes, interoperability is not a yes-or-no proposition but has shades of meaning.

Interoperability General Scenario



Portion of Scenario	Possible Values
Source	A system
Stimulus	A request to exchange information among system(s).
Artifact	The systems that wish to interoperate
Environment	System(s) wishing to interoperate are discovered at run time or known prior to run time.
Response	One or more of the following: • the request is (appropriately) rejected and appropriate entities (people or systems) are notified • the request is (appropriately) accepted and information is exchanged successfully • the request is logged by one or more of the involved systems
Response Measure	One or more of the following: • percentage of information exchanges correctly processed • percentage of information exchanges correctly rejected

Sample Concrete Interoperability Scenario



- Our vehicle information system sends our current location to the traffic monitoring system.
- The traffic monitoring system combines our location with other information, overlays this information on a Google Map, and broadcasts it.
- Our location information is correctly included with a probability of 99.9%.

Goal of Interoperability Tactics

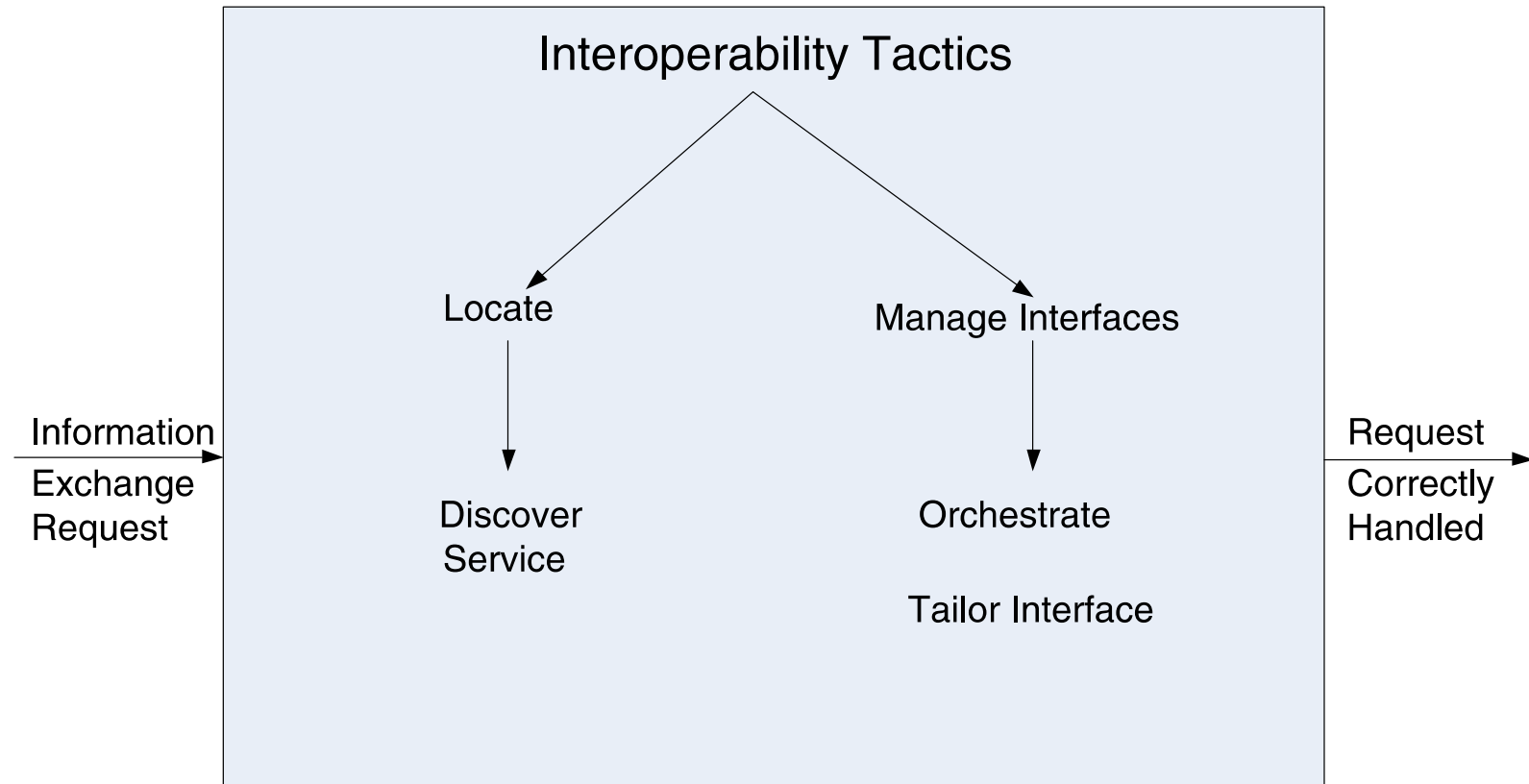


- For two or more systems to usefully exchange information they must
 - Know about each other. That is the purpose behind the locate tactics.
 - Exchange information in a semantically meaningful fashion. That is the purpose behind the manage interfaces tactics. Two aspects of the exchange are
 - Provide services in the correct sequence
 - Modify information produced by one actor to a form acceptable to the second actor.

Goal of Interoperability Tactics



Interoperability Tactics



Locate



- Discover service:
 - Locate a service through searching a known directory service.
 - There may be multiple levels of indirection in this location process
 - ✓ – i.e. a known location points to another location that in turn can be searched for the service.

Manage Interfaces



- Orchestrate: uses a control mechanism to coordinate, manage and sequence the invocation of services. Orchestration is used when systems must interact in a complex fashion to accomplish a complex task.
- Tailor Interface: add or remove capabilities to an interface such as translation, buffering, or data-smoothing.



Design Checklist for Interoperability

Allocation of Responsibilities

- Determine which of your system responsibilities will need to interoperate with other systems.
- Ensure that responsibilities have been allocated to detect a request to interoperate with known or unknown external systems
- Ensure that responsibilities have been allocated to
 - accept the request
 - exchange information
 - reject the request
 - notify appropriate entities (people or systems)
 - log the request (for interoperability in an untrusted environment, logging for non-repudiation is essential)



Design Checklist for Interoperability

Coordination Model

- Ensure that the coordination mechanisms can meet the critical quality attribute requirements. Considerations for performance include:
- Volume of traffic on the network both created by the systems under your control and generated by systems not under your control.
- Timeliness of the messages being sent by your systems
- Currency of the messages being sent by your systems
- Jitter of the messages arrival times.
- Ensure that all of the systems under your control make assumptions about protocols and underlying networks that are consistent with the systems not under your control.

Design Checklist for Interoperability



Data Model

- Determine the syntax and semantics of the major data abstractions that may be exchanged among interoperating systems.
- Ensure that these major data abstractions are consistent with data from the interoperating systems. (If your system's data model is confidential and must not be made public, you may have to apply transformations to and from the data abstractions of systems with which yours interoperates.)

Design Checklist for Interoperability



Mapping Among Architectural Elements

- For interoperability, the critical mapping is that of components to processors. Beyond the necessity of making sure that components that communicate externally are hosted on processors that can reach the network, the primary considerations deal with meeting the security, availability, and performance requirements for the communication.
- These will be dealt with in their respective chapters.



Design Checklist for Interoperability

Resource Management

- Ensure that interoperation with another system (accepting a request and/or rejecting a request) can never exhaust critical system resources (e.g., can a flood of such requests cause service to be denied to legitimate users?).
- Ensure that the resource load imposed by the communication requirements of interoperation is acceptable.
- Ensure that if interoperation requires that resources be shared among the participating systems, an adequate arbitration policy is in place.



Design Checklist for Interoperability

Binding Time

- Determine the systems that may interoperate, and when they become known to each other. For each system over which you have control
- Ensure that it has a policy for dealing with binding to both known and unknown external systems.
- Ensure that it has mechanisms in place to reject unacceptable bindings and to log such requests.
- In the case of late binding, ensure that mechanisms will support the discovery of relevant new services or protocols, or the sending of information using chosen protocols.



Design Checklist for Interoperability

Choice of Technology

- For any of your chosen technologies, are they “visible” at interface boundary of a system? If so, what interoperability effects do they have? Do they support, undercut, or have no effect on the interoperability scenarios that apply to your system? Ensure the effects they have are acceptable.
- Consider technologies that are designed to support interoperability, e.g. Web Services. Can they be used to satisfy the interoperability requirements for the systems under your control?

Summary



- Interoperability refers to the ability of systems to usefully exchange information.
- Achieving interoperability involves the relevant systems locating each other and then managing the interfaces so that they can exchange information.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Testability

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs



SEZG651/ SSZG653

Software Architectures

Module 2-CS05A

Chapter Outline



What is Testability?

Testability General Scenario

Tactics for Testability

A Design Checklist for Testability

Summary



What is Testability?

Software testability refers to the ease with which software can be made to demonstrate its faults through (typically execution-based) testing.

Specifically, testability refers to the probability, assuming that the software has at least one fault, that it will fail on its *next* test execution.

If a fault is present in a system, then we want it to fail during testing as quickly as possible.

What is Testability?



For a system to be properly testable, it must be possible to *control* each component's inputs (and possibly manipulate its internal state) and then to *observe* its outputs (and possibly its internal state).

Testability General Scenario



Portion of Scenario	Possible Values
Source	Unit testers, integration testers, system testers, acceptance testers, end users, either running tests manually or using automated testing tools
Stimulus	A set of tests are executed due to the completion of a coding increment such as a class, layer or service; the completed integration of a subsystem; the complete implementation of the system; or the delivery of the system to the customer.
Environment	Design time, development time, compile time, integration time, deployment time, run time
Artifacts	The portion of the system being tested
Response	One or more of the following: execute test suite and capture results; capture activity that resulted in the fault; control and monitor the state of the system
Response Measure	One or more of the following: effort to find a fault or class of faults, effort to achieve a given percentage of state space coverage; probability of fault being revealed by the next test; time to perform tests; effort to detect faults; length of longest dependency chain in test; length of time to prepare test environment; reduction in risk exposure ($\text{size}(\text{loss}) * \text{prob}(\text{loss})$)

Sample Concrete Testability Scenario



- The unit tester completes a code unit during development and performs a test sequence whose results are captured and that gives 85% path coverage within 3 hours of testing.

Goal of Testability Tactics

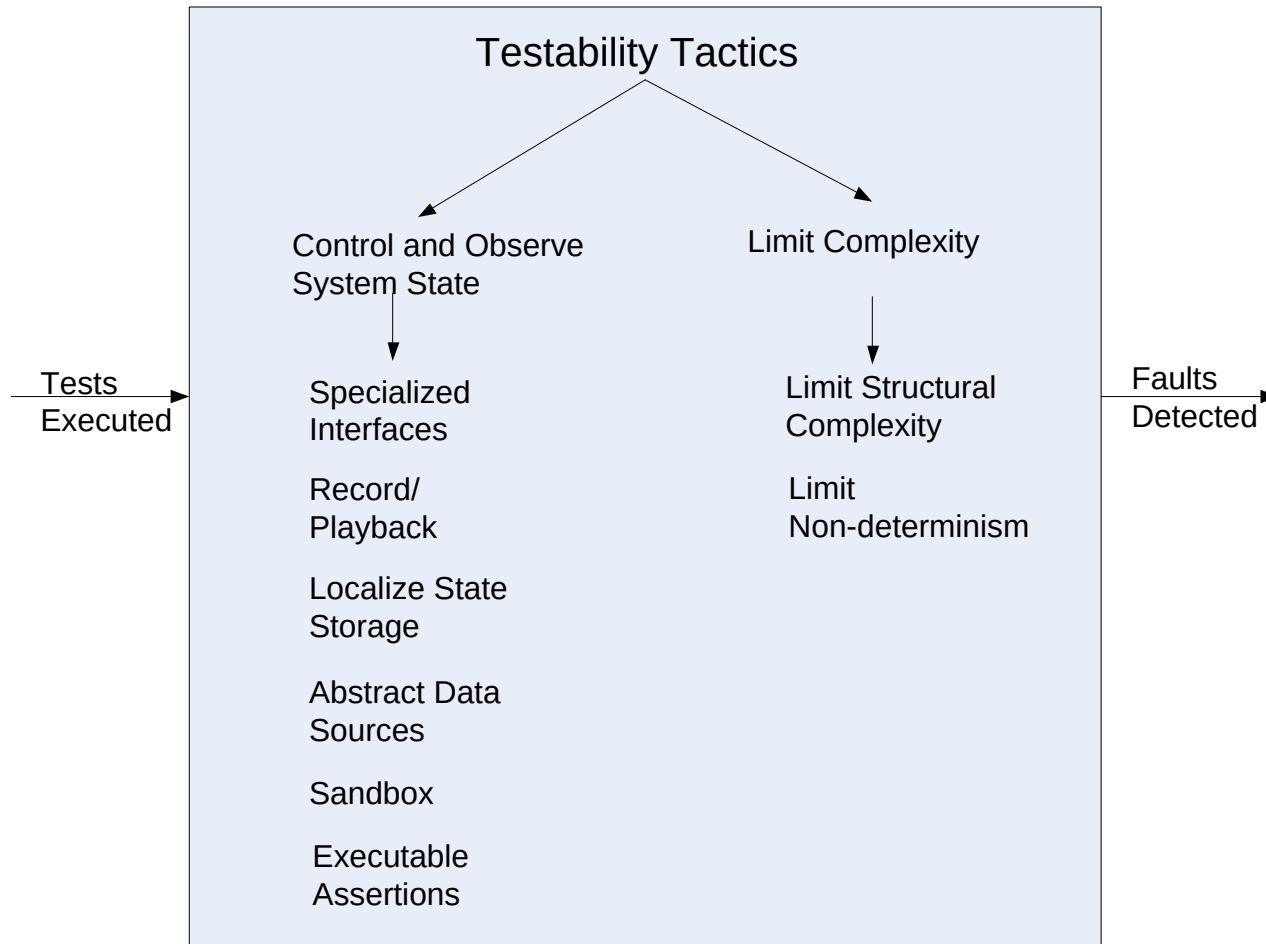


- The goal of tactics for testability is to allow for easier testing when an increment of software development has completed.
- Anything the architect can do to reduce the high cost of testing will yield a significant benefit.
- There are two categories of tactics for testability:
 - The first category deals with adding controllability and observability to the system.
 - The second deals with limiting complexity in the system's design.

Goal of Testability Tactics



Testability Tactics



Control and Observe System State



- Specialized Interfaces: to control or capture variable values for a component either through a test harness or through normal execution.
- Record/Playback: capturing information crossing an interface and using it as input for further testing.
- Localize State Storage: To start a system, subsystem, or module in an arbitrary state for a test, it is most convenient if that state is stored in a single place.

Control and Observe System State



- Abstract Data Sources: Abstracting the interfaces lets you substitute test data more easily.
- Sandbox: isolate the system from the real world to enable experimentation that is unconstrained by the worry about having to undo the consequences of the experiment.
- Executable Assertions: assertions are (usually) hand coded and placed at desired locations to indicate when and where a program is in a faulty state.

Limit Complexity



- Limit Structural Complexity: avoiding or resolving cyclic dependencies between components, isolating and encapsulating dependencies on the external environment, and reducing dependencies between components in general.
- Limit Non-determinism: finding all the sources of non-determinism, such as unconstrained parallelism, and weeding them out as far as possible.

Design Checklist for Testability



Allocation of Responsibilities

Determine which system responsibilities are most critical and hence need to be most thoroughly tested.

Ensure that additional system responsibilities have been allocated to do the following:

- execute test suite and capture results (external test or self-test)
- capture (log) the activity that resulted in a fault or that resulted in unexpected (perhaps emergent) behavior that was not necessarily a fault
- control and observe relevant system state for testing

Make sure the allocation of functionality provides high cohesion, low coupling, strong separation of concerns, and low structural complexity.

Design Checklist for Testability



Coordination Model

Ensure the system's coordination and communication mechanisms:

- support the execution of a test suite and capture of the results within a system or between systems
- support capturing activity that resulted in a fault within a system or between systems
- support injection and monitoring of state into the communication channels for use in testing, within a system or between systems
- do not introduce needless non-determinism

Design Checklist for Testability



Data Model

Determine the major data abstractions that must be tested to ensure the correct operation of the system.

- Ensure that it is possible to capture the values of instances of these data abstractions.
- Ensure that the values of instances of these data abstractions can be set when state is injected into the system, so that system state leading to a fault may be re-created.
- Ensure that the creation, initialization, persistence, manipulation, translation, and destruction of instances of these data abstractions can be exercised and captured

Design Checklist for Testability



Mapping Among Architectural Elements	Determine how to test the possible mappings of architectural elements (especially mappings of processes to processors, threads to processes, modules to components) so that the desired test response is achieved and potential race conditions identified. In addition, determine whether it is possible to test for illegal mappings of architectural elements.
---	--

Design Checklist for Testability



Resource Management

Ensure there are sufficient resources available to execute a test suite and capture the results.

Ensure that your test environment is representative of (or better yet, identical to) the environment in which the system will run.

Ensure that the system provides the means to:

- test resource limits
- capture detailed resource usage for analysis in the event of a failure
- inject new resources limits into the system for the purposes of testing
- provide virtualized resources for testing

Design Checklist for Testability



Binding Time

Ensure that components that are bound later than compile time can be tested in the late bound context.

Ensure that late bindings can be captured in the event of a failure, so that you can re-create the system's state leading to the failure.

Ensure that the full range of binding possibilities can be tested.

Design Checklist for Testability



Choice of Technology

Determine what technologies are available to help achieve the testability scenarios that apply to your architecture. Are technologies available to help with regression testing, fault injection, recording and playback, and so on?

Determine how testable the technologies are that you have chosen (or are considering choosing in the future) and ensure that your chosen technologies support the level of testing appropriate for your system. For example, if your chosen technologies do not make it possible to inject state, it may be difficult to re-create fault scenarios.

Summary



- Ensuring that a system is easily testable has payoffs both in terms of the cost of testing and the reliability of the system.
- Controlling and observing the system state are a major class of testability tactics.
- Complex systems are difficult to test because of the large state space in which their computations take place, and because of the larger number of interconnections among the elements of the system. Consequently, keeping the system simple is another class of tactics that supports testability.